

What is QPS and How it Affects System Design

 systemdesignschool.io/fundamentals/qps



QPS and System Design

Let's talk more about QPS since it's typically the bottleneck in system design. First, let's clarify the terminologies.

RPS vs QPS

Requests Per Second (RPS) and Queries Per Second (QPS) are metrics commonly used to measure the performance of systems that handle incoming requests, particularly in web services and databases.

- **RPS:** RPS refers to the total number of requests received by a server in one second. This metric encompasses all types of requests, including those for static resources (like images) and dynamic content (like API calls).
- **QPS:** QPS specifically measures the number of database queries executed per second. It focuses on the interactions with the database layer, indicating how many queries are processed within a second.

[Stateless services](#) that handles web requests are easier to scale. Therefore, databases are often the most constrained part of the system, so engineers monitor QPS closely to ensure they don't overload the system. People often say QPS when they really mean RPS. For simplicity, we'll use QPS here.

QPS and System Design

Now let's get a feel of what we need to handle different levels of QPS.

Low QPS: 1-100 QPS

Typical Scenarios: Small startup applications, early-stage products, small-scale internal tools, or local community websites.

At this level, a traditional **monolithic architecture** can often suffice. Such systems don't need to worry about large volumes of data or excessive amounts of traffic, allowing for simpler infrastructure. Your classic MySQL + PHP stack would do.

Technology Types and Examples:

- Backend Architecture: A simple, unified backend structure can efficiently handle this load without unnecessary complexity. For example, frameworks like Django (Python) or Express.js (Node.js) are suitable.
- Database: A straightforward, relational database setup works well to manage data consistently, such as PostgreSQL or MySQL.
- Infrastructure: A **single instance** or server is usually adequate to support the entire application, such as using a cloud instance like AWS EC2 or DigitalOcean Droplet.

Example: A local online store with a few hundred customers will typically have low QPS, and a simple monolithic app can efficiently manage it without any sophisticated scaling strategy.

Medium QPS: 100-1,000 QPS

Typical Scenarios: Growing SaaS applications, popular blogs, or community-driven platforms.

When the QPS grows, a monolithic architecture may struggle to meet demand, and performance issues may arise due to database bottlenecks or insufficient computing resources. At this stage, **horizontal scaling** and **decoupling** become more important.

Technology Types and Examples:

- Backend Architecture: A modular approach, such as **microservices**, allows individual components to scale independently, addressing bottlenecks effectively.
- Database Strategy: Techniques like **sharding, replication, or caching** help manage higher demand on data storage systems. Examples include read replicas for MySQL or using Redis for caching.
- Infrastructure Management: Horizontal scaling with containerized services and orchestration tools ensures flexibility and resiliency. Tools like Docker and Kubernetes can manage these services.

Example: A small social networking site that handles more frequent user interactions, like posting or liking content, might reach a medium QPS level.

High QPS: 1,000-100,000 QPS

Typical Scenarios: Large e-commerce sites, popular SaaS products, or social media platforms.

At this level, services need to be highly scalable and resilient to handle a surge of incoming requests. **Microservices, message queues, redundancy, and distributed systems** are commonly adopted to meet these needs.

Technology Types and Examples:

- Backend Architecture: Fully embracing **microservices**, where components can be independently deployed and scaled, becomes crucial. Efficient inter-service communication is needed to maintain performance.
- Event-driven architecture: Use **message queues** (e.g., Kafka) to buffer the incoming requests and to decouple components.
- Data Management: Use **distributed data stores** that are built to scale horizontally, and implement message queues or data streaming systems for real-time processing. Examples include Apache Kafka for streaming and Cassandra or MongoDB for distributed databases.
- In-memory caching: Heavy caching to reduce load on databases (e.g., caching search results)
- Infrastructure Management: Container orchestration and advanced load balancing to efficiently distribute traffic are necessary at this level. Kubernetes and AWS Elastic Load Balancing are commonly used.

Example: An e-commerce platform like Etsy or Shopify handles a high number of concurrent users, especially during sales events. To manage this, a combination of distributed databases, in-memory caching, and scalable microservices can handle the high volume of interactions seamlessly.

Very High QPS: 100,000+ QPS

Typical Scenarios: Global-scale platforms such as major social media sites, financial trading systems, or large streaming services.

For this scale, even small inefficiencies can lead to severe issues. The architecture has to be extremely robust, distributed, and designed to be resilient to failures.

Technology Types and Examples:

- Globally replicated databases: Data is replicated across regions to reduce latency
- Multi-region deployments: Applications run in multiple data centers or cloud regions for resilience
- Edge computing: Run code closer to the users at edge locations (e.g., on CDNs)
- Serverless architecture: Automatically scale functions to handle unpredictable loads
- Multi-cloud environments: Use resources from multiple cloud providers for redundancy

Example: Netflix, which has millions of users streaming content simultaneously, employs a highly distributed architecture with extensive caching at various layers and relies heavily on CDNs to ensure smooth playback.